CMPE 660 Laboratory Exercise 3

AXI and Memory Interfaces

Diego Diaz Peon Gonzalez Pacheco
Performed: 10/28/2026
Submitted: 11/03/2026

TA: Anthony Doan
Drew Ryczek
Dominik Ramut

Lecture Section: 1
Professor: Dr. Lukowiak

By submitting this report, you attest that you neither have given nor have received any assistance (including writing, collecting data, plotting figures, tables or graphs, or using previous student reports as a reference), and you further acknowledge that giving or receiving such assistance will result in a failing grade for this course.

Your Signature: _____

Abstract

In this exercise, the UART and AXI4-Stream FIFO system from Exercise 2 was connected to memory through an AXI4-Lite interface on a Nexys4 DDR FPGA. The exercise was split into three parts. In the first part, a Stream-to-Lite state machine was designed that took bytes from the receive FIFO, wrote four of them to a Block RAM through its AXI4-Lite interface, read them back, and sent them to the transmit FIFO so they were returned to the PC. In the second part, a file-driven testbench was written that read `TEST` and `WAIT` commands from a text file, sent any number of bytes to the design, checked the bytes that came back, and logged them to an output file. In the third part, the Block RAM was replaced by the on-board DDR2 memory, using the Xilinx Memory Interface Generator (MIG) and an AXI Protocol Converter, while the `top` module kept the same AXI4-Lite interface. The DDR2 simulation exposed a handshake bug that the Block RAM had hidden: the MIG accepts the write address and the write data in different clock cycles, which the original state machine did not handle. After fixing it, all three parts passed in simulation and on the FPGA.

1 Design Methodology

1.1 AXI4-Lite Protocol

AXI4-Lite is a memory-mapped bus with five independent channels: write address (AW), write data (W), write response (B), read address (AR), and read data (R). Each channel has its own `valid` and `ready` signals, and a transfer on a channel only happens on a clock edge where both are high, the same rule as in AXI-Stream. A write needs one transfer on AW, one on W, and then a response on B. A read needs one transfer on AR, followed by the data on R. Unlike AXI-Stream, every transfer has an address, and AXI4-Lite only allows single transfers, with no bursts.

Three rules from the AXI specification shaped the design. A master must hold `valid` high, with its data unchanged, until the slave raises `ready`. A master may tie `bready` and `rready` high if it can always accept a response. And the slave is allowed to accept the write address and the write data in any order, in the same cycle or in different cycles. The last rule turned out to matter in Part 3.

1.2 Common Design Elements

The UART wrappers (`AXIS_UART_RX` and `AXIS_UART_TX`) and the two AXI4-Stream Data FIFO IP cores were reused from Exercise 2. As before, a counter, `scount12`, created a one-clock enable pulse, `sclk_en_16_x_baud`, at 16 times the baud rate of 115,200, so the whole design ran on one clock. The receive FIFO, `axis_data_fifo_recieve`, had a depth of 512, and the transmit FIFO, `axis_data_fifo_transmit`, had a depth of 1024. The FIFOs' active-low reset, `aresetn`, was driven by `not rst`.

All of the new logic was placed in `top.vhd`, which has the UART pins and an AXI4-Lite master port. The memory itself was placed in a separate wrapper, `fpga_top_BRAM` in Parts 1 and 2 and `fpga_top` (`fpga_top_DDR2.vhd`) in Part 3. Because both wrappers connect to the same AXI4-Lite port, the same `top.vhd` could be used with either memory.

Each part also needed a constraints (XDC) file mapping `clk_100MHz`, `sys_rst`, `i_UART_RX`, and `o_UART_TX` to their pins. Every port needed an `IOSTANDARD` of `LVCMOS33`, otherwise bitstream generation failed with a DRC NSTD-1 error. In Part 3, the DDR2 pins were constrained by the MIG itself.

1.3 Part 1: AXI4-Lite and Block RAM

For the first part, the `top.vhd` skeleton and the `fpga_top_BRAM` wrapper were given. A Block Memory Generator IP was created with an AXI4-Lite interface, so the Block RAM controller was included inside the IP. The Stream-to-Lite logic was then designed to sit between the two FIFOs and the Block RAM. An RTL schematic of the full design is shown in Figure 1.

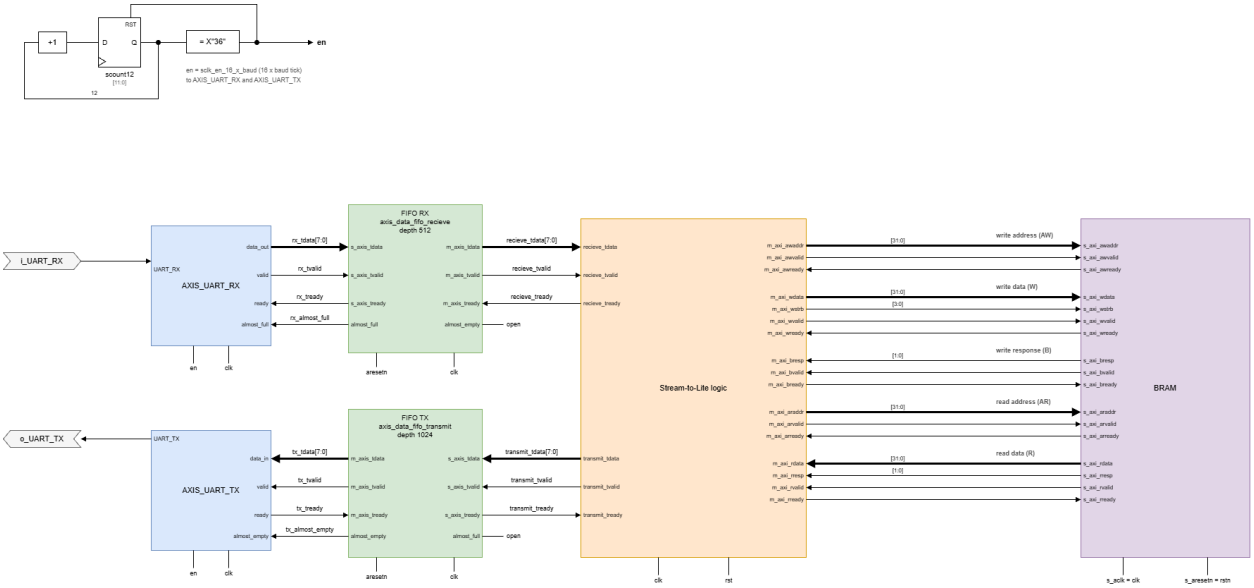


Figure 1: RTL schematic for the UART to Block RAM system

Since AXI-Stream has no address, the Stream-to-Lite logic had to generate the addresses itself. It used a state machine with six states, `idle`, `write`, `bresp`, `read`, `data`, and `send`, shown in Figure 2. In `idle`, the state machine waited for a byte from the receive FIFO. When `recieve_tvalid` was high, `recieve_tready` was raised for that clock, so the FIFO popped the byte on the same edge that the byte was loaded into the data register `tdata`, and the state changed to `write`. In `write`, `m_axi_awvalid` and `m_axi_wvalid` were raised until the memory accepted the write, and in `bresp` the state machine waited for `m_axi_bvalid`. After four writes, it moved to `read`, where `m_axi_arvalid` was raised until `m_axi_arready`. In `data`, it waited for `m_axi_rvalid` and loaded the low byte of `m_axi_rdata` into the register `transmit_tdata`, since `rvalid` only lasts one clock. In `send`, `transmit_tvalid` was raised until the transmit FIFO accepted the byte. After four reads, the state machine returned to `idle`.

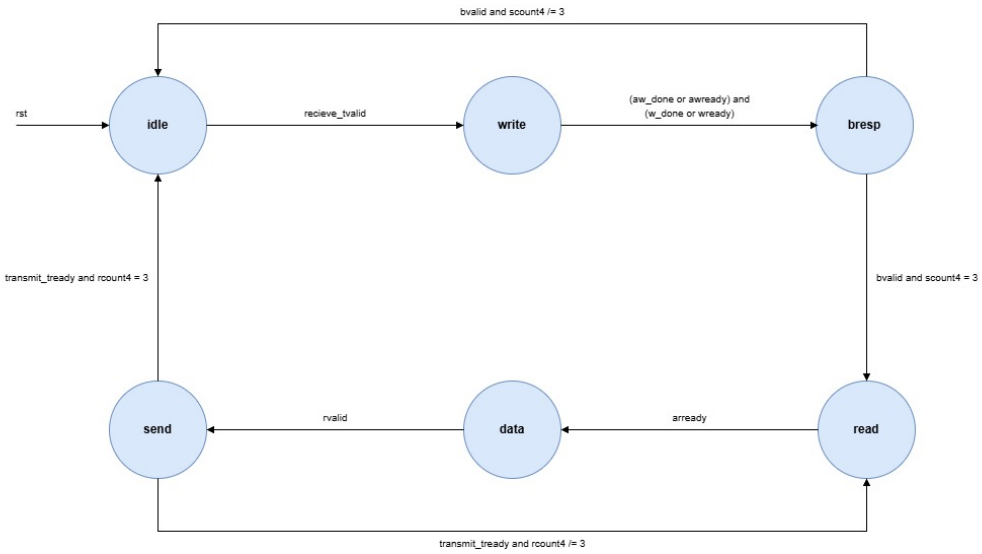


Figure 2: State diagram for the Stream-to-Lite logic in Part 1

The addresses came from two 4-bit counters, `scount4` for writes and `rcount4` for reads. `scount4` counted each write response, when the state was `bresp` and `m_axi_bvalid` was high, and `rcount4` counted each byte sent, when the state was `send` and `transmit_tready` was high. Both wrapped back to 0 after the fourth transfer, so the next group of four bytes used the same addresses. Since each address points to one byte and every word is 4 bytes, each counter was shifted left by two bits, giving the addresses 0x0, 0x4, 0x8, and 0xC:

```
m_axi_awaddr <= x"000000" & "00" & std_logic_vector(scount4) & "00";
```

Each byte was written into the low byte of the 32-bit word, with `m_axi_wdata` set to `x"000000" & tdata` and `m_axi_wstrb` set to `"0001"`, so only that byte lane was written. `m_axi_bready` and `m_axi_rready` were tied to `'1'`, since the state machine could always accept a response, and `m_axi_bresp` and `m_axi_rresp` were not used.
Following the professor's recommendation, each state machine output was driven by its own combinational process with a `case` on the state, similar to the output process in `MLUART_RX`. This made every `valid` change on the same clock edge as the state, instead of one clock late, which would have held `valid` high for one clock after the handshake and caused a second transfer. It also meant every `valid` was `'0'` while `rst` held the state machine in `idle`, as the AXI specification requires. The two data registers, `tdata` and `transmit_tdata`, were clocked processes so they would not become latches.

1.4 Part 2: File-Based Testbench

For the second part, the testbench was changed to read its stimulus from a text file, `stim_in.txt`, so that new tests could be added without changing the VHDL. Each line of the file was read with `readline`, and the first four characters were read into a 4-character string and compared in a `case` statement. Two commands were supported:

- `TEST <XX> <XX> ...`: send each hex byte to the design over UART, then receive the same number of bytes back and check them.

- `WAIT <n>`: wait for `n` rising edges of the 100 MHz clock.

Any other command was reported as an error. For `TEST`, the bytes were read one at a time with the three-argument version of `hread`, which returns `false` in its third argument when there are no more values on the line. This allowed any number of bytes on a line. The bytes were stored in an array of up to 16 bytes and then sent with the given `send_char` procedure. Each received byte was written to the output file `recieved.txt` together with the simulation time, using `write(L_OUT, now)` and `hwrite`, and an `assert` checked that it matched the byte that was sent. The stimulus file used for the results was:

```
TEST A2 B3 C5 D8
WAIT 100
TEST 25 8D 6F B4
```

One timing detail in the testbench had to be handled. The FPGA's receiver finishes a byte partway through its stop bit, and with the Block RAM the write and the read take only a few clocks, so the echo of the first byte starts about 5 µs before `send_char` returns from the last byte. If `rx_char` was only called after that, it missed the start of the echo. To fix this, a `last` flag was added to `send_char`, and for the last byte of a `TEST` the final `wait for symbol_len` of the stop bit was skipped. The stop bit itself was still driven high, so the frame on `i_UART_RX` was unchanged.
The design was also changed to support a configurable number of bytes. The hard-coded compare value `x"3"` in the counters and the state machine was replaced by a constant, `n_bytes`, set to 4, and the compares were written as `to_unsigned(n_bytes - 1, scount4'length)`.

1.5 Part 3: MIG and DDR2 Memory

For the third part, the Block RAM was replaced by the Nexys4 DDR's 128 MB DDR2 memory. The wrapper `fpga_top_DDR2.vhd` was given, and three IP cores were generated to match its components:

- `clk_wiz_0`, a Clocking Wizard that creates the 200 MHz reference clock the MIG needs from the 100 MHz board clock, with an active-high `reset` and a `locked` output.
- `mig_7series_0`, the Memory Interface Generator, configured for DDR2 with an AXI4 slave interface (32-bit data, 27-bit address) following the Memory tutorial, with the DDR2 pins imported from `mig.ucf`.
- `axi_protocol_converter_0`, which converts the AXI4-Lite signals from `top` into full AXI4 for the MIG, with a 27-bit address and 32-bit data.

The RTL schematic of the system is shown in Figure 3. Between the converter and the MIG, the AXI4 signals of each channel are grouped into one bus. Full AXI4 adds signals for bursts, such as `awlen`, `awsize`, `awburst`, and `wlast`, which the converter sets for single transfers.

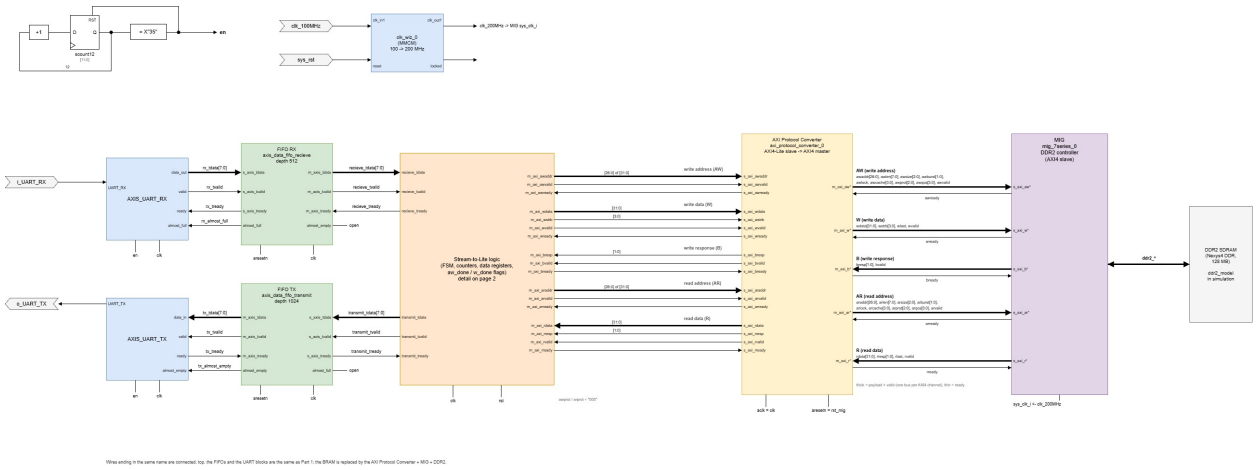


Figure 3: RTL schematic for the UART to DDR2 system

The biggest difference from Part 1 was the clocking. The MIG generates its own user interface clock, `ui_clk`, and in this wrapper `ui_clk` is the `clk` for `top`, both FIFOs, and the converter. The MIG's synchronized reset, `ui_clk_sync_rst`, became `rst_mig`, which reset `top`. The `locked` output of the Clocking Wizard drove the MIG's active-low system reset, so the MIG stayed in reset until its reference clock was stable. Since the baud rate counter in `top` assumes a 100 MHz clock, the frequency of `ui_clk` had to be 100 MHz for the UART to work.

For simulation, the given DDR2 model, `ddr2_model.v` and `ddr2_model_parameters.vh`, was added as a simulation source and connected to the DDR2 pins in `tb_fpga_top_DDR2`. The stimulus process from Part 2 was copied into this testbench without changes. Because the MIG and the DDR2 model make the simulation very slow, `log_wave -r /` was run in the Tcl console before each run, so any signal could be added to the waveform afterwards without running again.

The `top.vhd` module was first used without changes, but the DDR2 simulation returned the wrong data. The cause and the fix are described in Section 2.1.3. The fix added two flags to the Stream-to-Lite logic, `aw_done` and `w_done`. Since the change only follows the AXI rules more closely, the final `top.vhd` also works with the Block RAM.

2 Results and Analysis

2.1 Simulation Results

2.1.1 Part 1: Block RAM

The simulation of Part 1 is shown in Figure 4. The testbench sent four bytes back-to-back on `i_UART_RX`: 0xA2, 0xB4, 0xC5, and 0x32, which took about 340 µs. As each byte arrived, it was loaded into `tdata` and written to the Block RAM, and `s_axi_wdata` shows each byte in the low 8 bits of the word. Since the write counter only increments after the write response, `s_axi_awaddr` in the waveform already shows the address for the next byte: 0x4 after 0xA2 was written to 0x0, 0x8 after 0xB4, 0xC after 0xC5, and back to 0x0 after 0x32 was written

to 0xC. Once the fourth write finished, at about 341 µs, the four words were read back from addresses 0x0 to 0xC, the last one being 0x00000032 on `s_axi_rdata`, and the four bytes were sent on `o_UART_TX` in the same order they were received. All four matched, so no `assert` failed.

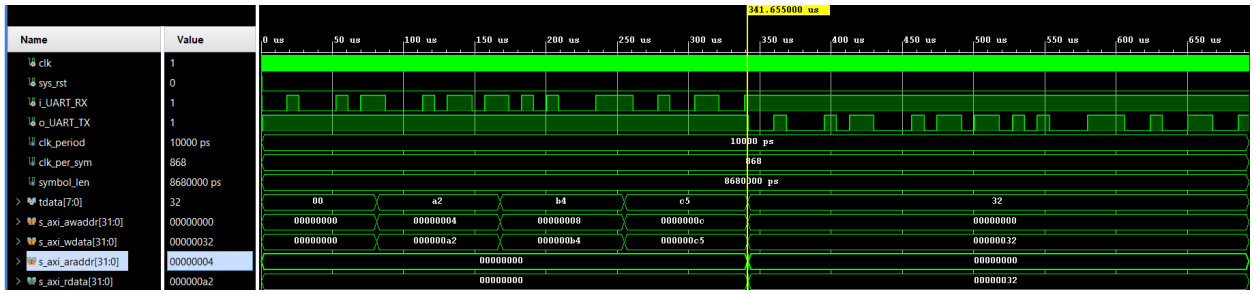


Figure 4: Behavioral simulation of the UART to Block RAM system

2.1.2 Part 2: File-Based Testbench

The simulation of Part 2 is shown in Figure 5, using the stimulus file `stim_in.txt` listed in the Part 2 methodology. In the first test, 0xA2, 0xB3, 0xC5, and 0xD8 were sent and written to the Block RAM, and `s_axi_rdata` then showed them being read back, ending with 0x000000D8. After the 100-cycle `WAIT`, the second test sent 0x25, 0x8D, 0x6F, and 0xB4, which were written to the same four addresses and read back, ending with 0x000000B4. Every byte returned on `o_UART_TX` matched the file, so no `assert` failed, and each received byte was logged to `recieved.txt`. The `WAIT` itself only adds 1 µs between the tests, which is too short to see at this scale, since the second test starts as soon as the last byte of the first test has been received. Most of the time between the last byte of one test and the first byte of the next is UART time: about 338.5 µs to send the four bytes and about 90 µs to receive the first one back.

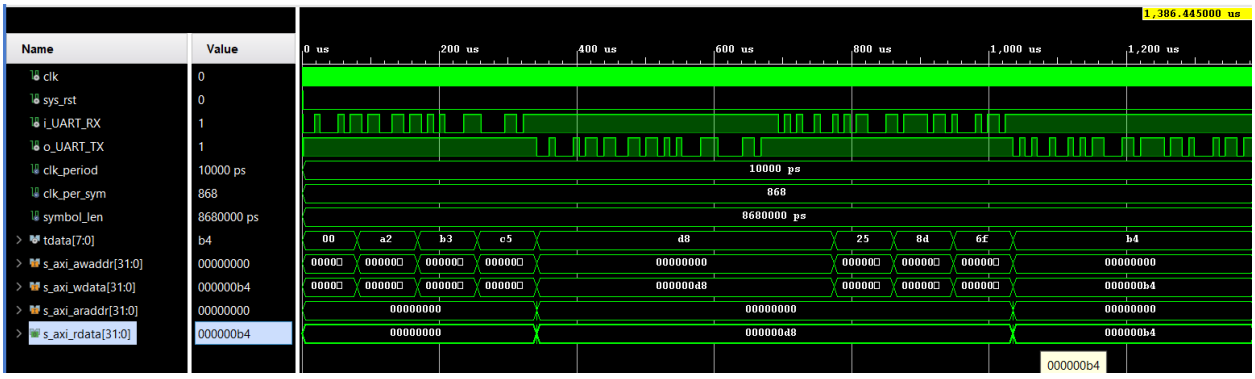


Figure 5: Behavioral simulation of the file-based testbench with two tests

2.1.3 Part 3: DDR2

The first DDR2 simulation, using the unchanged `top.vhd` from Part 1, returned B3, C5, D8, and 22 instead of A2, B3, C5, and D8. The first byte was missing, the next three came

back one position early, and the last byte was 0x22. Two startup problems were ruled out first. `rst_mig` went low almost at the start of the simulation, so `top` was out of reset before the first byte arrived. The MIG's `init_calib_complete` signal, inside the MIG, showed that calibration of the DDR2 finished at about 77 µs. To be sure the first write did not overlap calibration, a `WAIT 20000` (200 µs) line was added at the start of `stim_in.txt`, but the result was the same.

Zooming in on the first write showed the real cause. While the state machine was in `write`, `m_axi_awready` pulsed high about four times and `m_axi_wready` three times, but never in the same clock until the end. The state machine only left `write` when both `ready` signals were high together, so it kept both `valid` signals high the whole time. Each `ready` pulse with `valid` still high was a complete handshake, so the MIG accepted the same address and the same data several times. This shifted which data went to which address, so the first byte was overwritten and the last address was never written, which explains the result above. With the Block RAM, `awready` and `wready` always went high in the same clock, so the bug never appeared in Parts 1 and 2. The AXI specification allows the slave to accept the two channels separately, so the original design depended on behavior the protocol does not guarantee.

To fix it, two clocked flags were added, `aw_done` and `w_done`. Each one is set when its own `ready` is high in the `write` state and cleared in `idle`. `m_axi_awvalid` is now `'1'` only in `write` while `aw_done` is `'0'`, and `m_axi_wvalid` only while `w_done` is `'0'`, so each `valid` drops as soon as its own handshake is done. The state machine leaves `write` once both channels have finished, with the condition

```
(aw_done = '1' or m_axi_awready = '1') and (w_done = '1' or m_axi_wready =
                                        '1').
```

The read side did not need this change, since a read only has one request channel, AR, and the state machine leaves `read` on the first `aready`.

After the fix, the DDR2 simulation returned the correct bytes for both tests. For Part 3, `stim_in.txt` started with a `WAIT 20000` line before the two tests from Part 2, so the first byte was sent after the DDR2 had finished calibrating. The output file is shown in Figure 6. The first byte arrived at about 630 µs, which is the 1.2 µs reset wait, the 200 µs `WAIT`, about 338.5 µs of sending, and about 90 µs to receive the first byte. The extra time the DDR2 takes for each write and read is less than a microsecond, so it is not visible next to the UART transfer times. The full simulation is shown in Figure 7. Each write now shows a single short pulse on `m_axi_awvalid` and `m_axi_wvalid`, and the DDR2 command lines show the refresh commands the MIG keeps sending while the design is idle. The bytes of each test arrived 88 µs apart, one UART frame plus a short gap, and the first byte of the second test was logged 429.55 µs after the last byte of the first, the same spacing as with the Block RAM in Part 2.

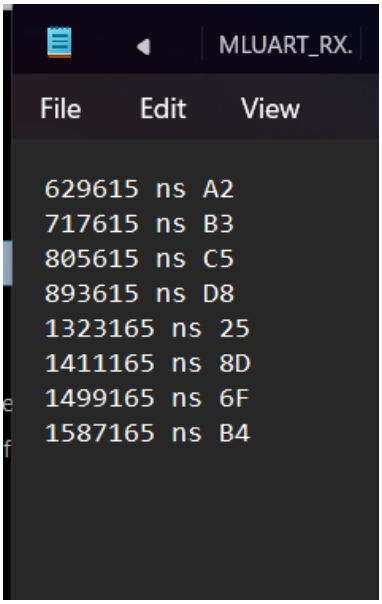


Figure 6: Output file `recieved.txt` from the DDR2 simulation

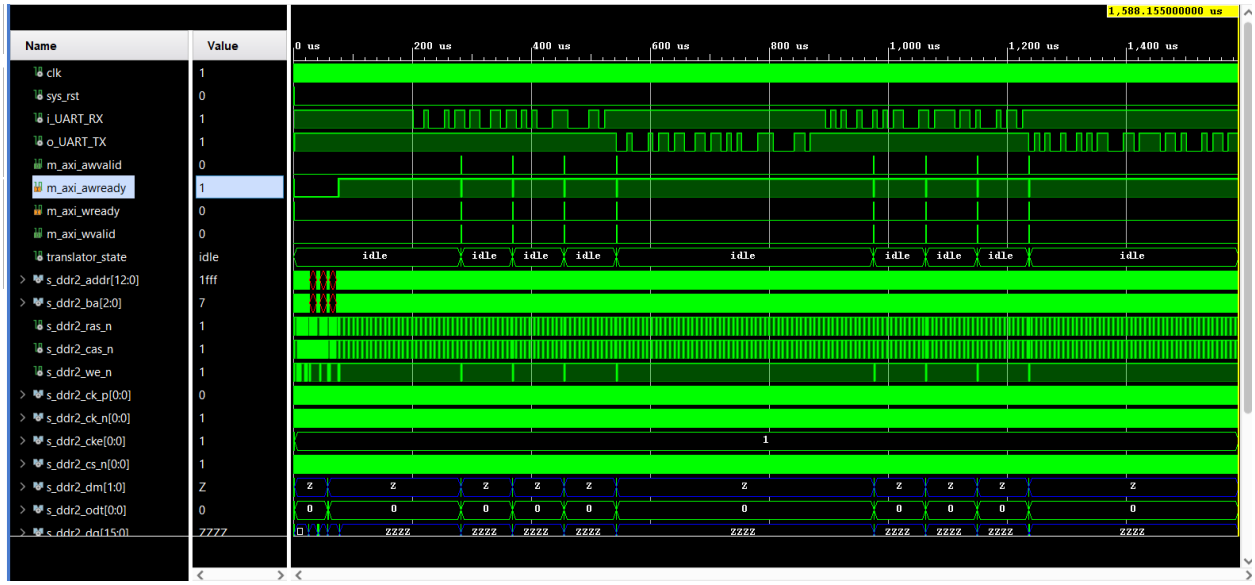


Figure 7: Behavioral simulation of the UART to DDR2 system after the fix

2.2 Hardware Demonstration

Parts 1 and 3 were programmed onto the Nexys4 DDR and tested from the PC at 115,200 baud, 8 data bits, no parity, and 1 stop bit. In both, every group of four bytes sent to the board was returned in the same order once all four had been written to memory, so from the PC the Block RAM and DDR2 versions looked exactly the same. The DDR2 version also has to calibrate after programming or after a reset, but this takes well under a millisecond, so it was never noticeable.

2.2.1 Part 1: Block RAM

The result of the Part 1 hardware test is shown in Figure 8.

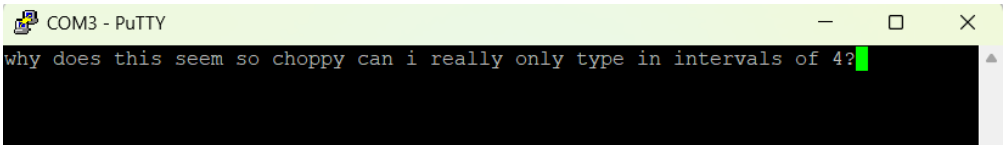


Figure 8: Part 1 hardware test with the Block RAM

2.2.2 Part 3: DDR2

For Part 3, the MATLAB script from Exercise 2 was used to send the Lena.pgm image through the DDR2 design at full speed, four bytes at a time. The image that came back was identical to the one that was sent, as shown in Figure 9.

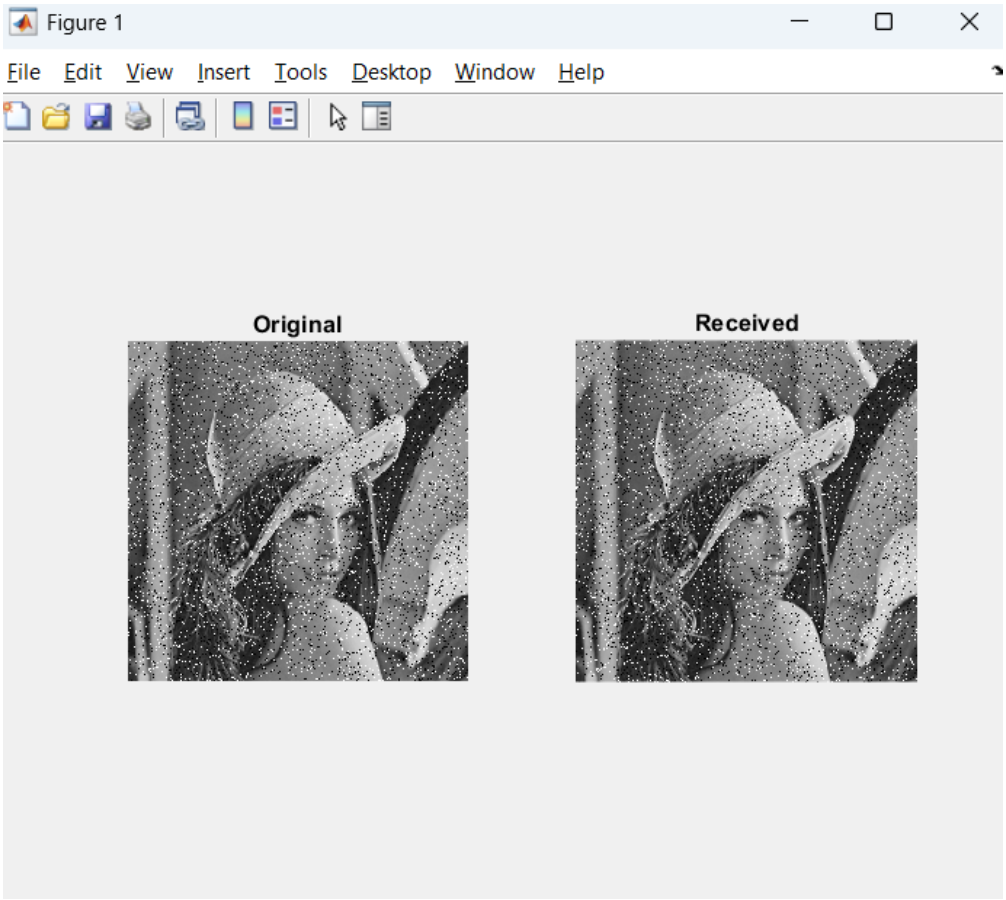


Figure 9: Lena.pgm before and after passing through the DDR2

3 Conclusion

In this exercise, an AXI4-Lite master was designed that turned a stream of bytes from a UART into memory writes and reads, and it was connected first to a Block RAM and then to DDR2 memory through the MIG and an AXI Protocol Converter. A file-based testbench was also written that reads `TEST` and `WAIT` commands, checks every byte that comes back, and logs the results, so new tests only need a new line in a text file. All three parts worked in simulation and on the FPGA.

Using a standard bus made the memory easy to replace. The same `top.vhd` worked with both memories because both sat behind the same AXI4-Lite interface, and only the wrapper changed. However, Part 3 showed that sharing an interface only works if the master follows the whole protocol and not just the behavior of the first slave it was tested with. The Block RAM always accepted the write address and the write data together, so a state machine that waited for both `ready` signals in the same clock passed every Block RAM test. The MIG accepted them separately, which the AXI specification allows, and the same state machine wrote the wrong data. Tracking each channel's handshake separately fixed the problem for both memories. Logging all waveforms with `log_wave -r /` was what made this bug practical to find, since the DDR2 simulation was too slow to run again for every signal that needed to be checked.

CMPE-660 Reconfigurable Computing

Exercise 3: AXI and Memory Interfaces

Student's Name: Diego Diaz Section:

Demo		Point Value	Points Earned	Date
Part 1: AXI/BRAM	Simulation	15	15	AD 10/23 ↓
	RTL + Hardware demo	15	15	
Part 2: Testbench	Simulation	20	20	10/29 DR
Part 3: DDR2	Simulation	15	15	10/29 DR ↓
	RTL + Hardware Demo	15	15	

To receive any grading credit students must earn points for both the demonstration and the report.

Report		Point Value	Points Earned	Comments
Report	Discussion	20		
Total for demo and report		100		

An electronic submission of project files via MyCourses is required. For information on how to properly submit the files, see the document "MyCoursesTut.pdf" on MyCourses.

Required file structure of tarball submission for this exercise:

```
Ex3
├── Part1
│   ├── [all necessary .vhd files]
│   └── Part1.xdc
├── Part2
│   └── [all necessary .vhd files]
└── Part3
    ├── [all necessary .vhd files]
    └── Part3.xdc
```

4

Figure 10: Demo sign-off sheet